# Reviewer Pack — Real Stable Minor Degree Bounds SOS Refutation Threshold for…

Entry 0cbc7a08f45d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 10:28:14 UTC

## Real Stable Minor Degree Bounds SOS Refutation Threshold for Max-CUT

**Entry ID**: `0cbc7a08f45d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 10:28:14 UTC

### 1. Conjecture

**Field A** (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

**Statement**:

> For any Max-CUT instance with n variables, if its moment matrix M has a real stable polynomial minor of degree d, then the SOS refutation threshold is Ω(d log n). Conversely, if the SOS refutation threshold is o(d log n), then M contains no real stable polynomial minor of degree d.

**Rationale (proposer's reasoning)**:

> Real stable polynomials encode geometric constraints on positive semidefinite matrices. Their minors capture the algebraic structure of the moment matrix, which directly influences the SOS hierarchy's ability to approximate Max-CUT. This linkage exposes the interplay between algebraic geometry and optimization complexity.

**Taxonomy category**: `SOS_DEGREE` (status at proposal time: partially_alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `f9541ae8e28b3ef0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> default: >=80% seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.95 | UNCERTAIN | SAFE |

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| NATURAL_PROOFS | SAFE | 0.95 | SAFE | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.00 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.95 | SAFE | SAFE |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - real stable polynomials Max-CUT SOS refutation threshold - moment matrix real stable polynomial Max-CUT SOS degree - SOS degree real stable minor Max-CUT refutation threshold

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.append((i, j))
        return edges

    def construct_moment_matrix(edges, n):
        M = [[0] * n for _ in range(n)]
        for i, j in edges:
            M[i][j] += 1
            M[j][i] += 1
        return M

    def is_positive_semidefinite(M):
        n = len(M)
        for k in range(1, n + 1):
            submatrix = [row[:k] for row in M[:k]]
            det = determinant(submatrix)
            if det < 0:
                return False
        return True

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
```

```
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += (-1) ** j * matrix[0][j] * determinant(submatrix)
    return det

def is_real_rooted(poly):
    n = len(poly)
    if n == 1:
        return poly[0] != 0
    for i in range(1, n):
        if poly[i].denominator != 1 or poly[i].numerator < 0:
            return False
    return True

def find_real_stable_minor(M, d=2):
    n = len(M)
    minors = []
    for i in range(n - d + 1):
        for j in range(i, n - d + 1):
            minor = [row[j:j+d] for row in M[i:i+d]]
            if is_positive_semidefinite(minor) and is_real_rooted([Fraction(coeff, 1) for coef
                minors.append(minor)
    return minors

def sos_refutation_threshold(M):
    n = len(M)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n):
        for j in range(i, n):
            A[i][j] = M[i][j]
            A[j][i] = M[i][j]
    A[n][n] = 1
    b = [0] * (n + 1)
    b[n] = -1
    x = gaussian_elimination(A, b)
    return sum(x[:n])

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            A[j][i] = 0
            for k in range(i+1, n+1):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
```

```
            x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
        return x

    def degree_2_moment_matrix(edges, n):
        M = [[0] * n for _ in range(n)]
        for i, j in edges:
            M[i][j] += 1
            M[j][i] += 1
        return M

    n = random.randint(5, 40)
    edges = generate_max_cut_instance(n)
    M = construct_moment_matrix(edges, n)

    minors = find_real_stable_minor(M, d=2)
    threshold = sos_refutation_threshold(degree_2_moment_matrix(edges, n))

    metric_value = len(minors) * threshold
    instances_tested = 1
    conjecture_holds = False
    counterexample = ""

    return {
        "metric_name": "sos_refutation_threshold",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_faili
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_edcb39f8.py", line 140, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_edcb39f8.py", line 118, in run_trial
    minors = find_real_stable_minor(M, d=2)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_edcb39f8.py", line 70, in find_real_sta
    if is_positive_semidefinite(minor) and is_real_rooted([Fraction(coeff, 1) for coeff in poly]):
                                                                                        ^^^^
NameError: name 'poly' is not defined
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> Test crashed due to undefined variable ‘poly’ before producing results | next: Debug the ‘find_real_stable_minor’ function’s polynomial extraction logic

## 11. Audit log (LLM calls)

**Total LLM calls**: 11

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | qwen3:8b | 0 | 0 | 136419 |
| 2 | propose | ollama_remote | qwen3:8b | 0 | 0 | 69218 |
| 3 | propose | ollama_remote | qwen3:8b | 0 | 0 | 71408 |
| 4 | preregistration | ollama_remote | qwen3:8b | 0 | 0 | 31808 |
| 5 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 27357 |
| 6 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 22585 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 23924 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 14030 |
| 9 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11786 |
| 10 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15870 |
| 11 | judge | ollama_remote | qwen3:8b | 0 | 0 | 21797 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 446203 ms total latency. Provider mix: {‘ollama_remote’: 11}

*(full prompt+response transcripts available in `research/audit/0cbc7a08f45d.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0cbc7a08f45d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/0cbc7a08f45d.tar.gz` (if generated)